بيانك
Bayanuk

Technical Plan

Architecture, stack, AI pipeline, API, security, testing, and deployment for the Bayanuk MVP.

Product	Bayanuk (بيانك), AI review of job offers and employment contracts
Owner	JX Tech Studio
Version	1.3 (updated for the integration contract)
Date	October 2026

Changes in v1.3

This version applies the approved frontend and backend integration contract (`docs/integration-contract.md`). Section 7 (API) is rewritten from it, and the sections that depend on it are aligned. Changed rows and lines are marked v1.3. The AI stack from v1.2 (Gemini 3.8 Flash, voyage-4, no AI models on our server) is unchanged.

Section	Change
4 Repositories and structure	One repository (`bayanuk`) with `backend/` and `frontend/`; working branch `development`.
5 Pipeline	Stop reasons stored as `status_detail` codes; the daily AI budget pauses processing instead of failing it.
6.5 Cost control	Duplicate upload and retry rules; what counts toward the daily limit; budget reset time.
7 API	Rewritten from the contract: conventions, guest token issued by the backend, sign-in flows, status and polling, duplicates, pagination, question count, signed URLs, and error codes.
8 to 10	Security controls for sign-in and guest tokens; frontend mock API; local development proxy; `development` branch in the Jenkins pipeline.

1. Purpose

This plan explains how Bayanuk is built: architecture, stack, the document processing pipeline, AI design, API, frontend structure, security, testing, and deployment. It is the technical reference for the backend (Laravel) and frontend (React) work. Product scope lives in the MVP Definition; the data model lives in the Database Schema; model and provider choices live in the AI Stack Decision; the exact requests and responses between frontend and backend live in the integration contract. v1.3

2. Architecture overview



Everything runs on the studio's own Ubuntu server with Docker Compose, the same way the studio already runs its other products. No AI model runs on our server. Only the masked text each task needs is sent to Gemini and Voyage AI; files, results, and embeddings stay on our server.

3. Technology stack

Layer	Choice	Why
Frontend	React + TypeScript, built with Vite	Team skill; fast builds; strict types catch API mismatches early.
UI	Tailwind CSS (logical properties for RTL) + shadcn/ui (Radix)	Accessible components; Radix supports RTL through a direction provider.
Data fetching	TanStack Query	Caching, polling of processing status, retries.
Routing / i18n	React Router, react-i18next	Arabic and English with dir switching.
Mock API	MSW (development only) v1.3	Frontend work continues before the backend is ready, against the contract.
Document viewer	react-pdf (pdf.js)	Show the original file and jump to the cited page.
Brand	Readex Pro font; colors from the logo kit	One family for Arabic and Latin.
Backend	Laravel (latest stable), PHP 8.3+	Team skill; mature queues, auth, storage, validation.
Auth	Laravel Sanctum (SPA cookie auth) + Socialite (Google) + custom magic link	No passwords to store or leak.
Queues	Redis + Laravel Horizon	The pipeline is asynchronous; Horizon gives visibility and retries.
AI integration	Prism (prism-php) behind our own AIService interface	One interface for all AI tasks; providers and models are configuration values.
AI models	Gemini 3.8 Flash (Gemini API, paid tier) for classification, extraction, chat, and OCR fallback; voyage-4 (Voyage AI) for embeddings	High quality in Arabic and English at low cost; no GPU server to run.
Database	PostgreSQL 16+ with pgvector	Relational data and vector search together.
File storage	RustFS (S3-compatible, self-hosted), Laravel S3 disk	Same as invoi; files never leave our server.
Text extraction	poppler pdftotext (spatie/pdf-to-text), PhpWord for DOCX	Fast and free for text PDFs.
OCR	Tesseract with Arabic + English data; Gemini reads the page image when OCR fails	Scanned documents; the API is used only for pages Tesseract cannot read.
Malware scan	ClamAV daemon	Every upload is scanned before processing.
CI/CD	Jenkins	Already used by the studio.

Layer	Choice	Why
Monitoring	Prometheus + Grafana, Laravel logs, Horizon dashboard	Already installed on the studio server.

4. Repositories and structure v1.3

One repository, `bayanuk`, in the JxTechStudio-org organization: `backend/` (Laravel API) and `frontend/` (React SPA). `Dockerfile`, `docker-compose.yml`, and `Jenkinsfile` sit at the repository root. Project documents live in `ProjectDocumentation/`, and the integration contract in `docs/integration-contract.md`. Branches: `main` (production), `development` (staging), and short-lived `feature/*` branches merged into `development` by pull request after review.

```
bayanuk/
  backend/              Laravel API
  frontend/             React SPA
  Dockerfile
  docker-compose.yml
  Jenkinsfile
```

frontend/ (feature-based)

```
src/
  app/                  router, providers (QueryClient, i18n, direction), layout
  features/
    upload/             UploadDropzone, file checks, upload mutation
    analysis/           ProcessingScreen, ResultDashboard, TermCards, FindingsList, Summary
    chat/               ChatPanel, SuggestedQuestions, CitationChip
    viewer/             DocumentViewer (react-pdf), page jump
    auth/               MagicLinkForm, GoogleButton, VerifyPage, CallbackPage
    documents/          MyDocuments list, delete dialog
    compare/            (MVP+) CompareView
  components/           shared UI (Button, Card, Badge, Dialog...) from shadcn/ui
  lib/                  api client (fetch wrapper), guest token, formatters
  mocks/                MSW handlers that follow the integration contract (development only)
  locales/              ar.json, en.json
```

backend/

```
app/
  Http/Controllers/Api/V1/          DocumentController, ChatController, AuthController,
                                    GuestTokenController, ...
  Http/Requests/                    UploadDocumentRequest, AskQuestionRequest, ...
  Http/Resources/                   DocumentResource, AnalysisResource, FindingResource
  Models/                           User, Document, DocumentPage, DocumentChunk, Analysis, ...
  Enums/                            DocumentStatus, DocumentType, FindingSeverity, ...
  Jobs/Pipeline/                    ScanFile, ExtractText, ClassifyDocument, AnalyzeDocument,
                                    DetectFindings, IndexChunks, DeleteDocument
  Services/AI/                      AIService (interface), PrismAIService, Prompts/, Schemas/
  Services/Extraction/              PdfTextExtractor, DocxTextExtractor, OcrExtractor, PiiMasker
  Services/Rules/                   FindingRules (config-driven), RuleEngine
  Policies/                         DocumentPolicy (owner or guest token)
config/bayanuk.php                  limits, retention, rule thresholds, model names, prices,
                                    daily AI budget
resources/prompts/                  versioned prompt templates (v1/classify.md, v1/extract.md ...)
tests/                              Feature/, Unit/, Golden/ (sample documents + expected JSON)
```

5. Document processing pipeline

Upload returns immediately with status `uploaded`. The rest runs as a chained queue batch; each job updates `documents.status` so the frontend can show progress. When processing stops or pauses, `documents.status_detail` holds one code that the frontend translates (section 7.5). v1.3

Step	Job	What it does	On failure
1	ScanFile	ClamAV scan; count pages (max 30). File type and size are already checked by content at upload.	`rejected` with `unsafe_file` or `too_many_pages`; file deleted. v1.3
2	ExtractText	pdftotext per page; if a page has almost no text or the Arabic text is broken, OCR it (Tesseract ara+eng) and mark is_scanned. If OCR still fails, send the page image to Gemini (extraction_method = vision). DOCX via PhpWord, split into logical pages. Clean text, normalize Arabic numerals, mask national ID, iqama, IBAN, phone numbers, and email.	`failed` with `file_unreadable`. v1.3
3	ClassifyDocument	Short Gemini call on the first pages only: job_offer, employment_contract, or other; detect language.	other leads to `rejected` with `not_employment_document`. v1.3
4	AnalyzeDocument	The only call that sends the full contract text, with the extraction JSON schema. Validate output; verify every quote exists in its page text; retry once on invalid output. Save analyses.result and extracted_terms. Generate summary.	`failed` after retry with `analysis_failed`; the technical cause stays in analyses.error_code. v1.3
5	DetectFindings	Run deterministic PHP rules on extracted_terms first, then AI checks for clauses rules cannot detect (non-compete, penalties, one-sided changes), using relevant chunks instead of the full text.	Analysis still shown without findings; flagged in logs.
6	IndexChunks	Split pages into chunks (about 500 tokens, 15% overlap, never across pages); create voyage-4 embeddings (1024); save to document_chunks.	Chat disabled for this document with a message; rest works.
7	Mark ready	status = ready; status_detail = null; processing_finished_at set; usage_events written (tokens and cost).	

Steps 5 and 6 run in parallel after step 4. Each job is idempotent (safe to retry) and has a timeout. Failed jobs are visible in Horizon and alert the team through Grafana. Processing stops at the first failure, so a document has one stop reason only.

Every job that calls an AI API checks the daily AI budget first. When the budget is reached, the document **pauses**; it does not fail. `status` stays at the last stage reached, `status_detail` becomes `ai_budget_reached`, and processing resumes on its own after the reset (00:00 Riyadh time), when `status_detail` goes back to null. v1.3

6. AI design

6.1 Provider abstraction

Controllers and jobs depend on an AIService interface with four methods: `classify()`, `extract()`, `answer()`, and `embed()`. The implementation uses Prism, so the provider and model for each task are configuration values. In the MVP, `classify()`, `extract()`, `answer()`, and the OCR fallback use Gemini 3.8 Flash, and `embed()` uses voyage-4. No code outside `Services/AI` knows which provider is used, so a provider can be replaced without changing the pipeline. A backup provider is still to be decided.

6.2 Structured output and anti-hallucination checks

- Extraction uses a JSON schema (structured output). The response is validated with Laravel validation rules before saving.
- Every value must come with a page number and a short quote. The backend checks that the quote actually appears in that page's text (after normalization). A value with a quote that cannot be found is downgraded to "not found".
- Totals are recomputed from components; a mismatch becomes a "Salary does not add up" finding instead of being silently corrected.
- The model is told to return "not found" instead of guessing. Prompts include Arabic and English examples.
- Prompts are files in the repository with a version number saved on each analysis, so results can be traced and compared after prompt changes.
- Staged analysis: the full contract is sent once, for extraction. Classification uses the first pages, findings use extracted terms and relevant chunks, and chat uses retrieved chunks only.
- Low temperature (0 to 0.2) for repeatable results.

6.3 Chat (retrieval-augmented generation)

1. Embed the question with the same embedding model (voyage-4).
2. Vector search inside the document's chunks only (top 6), plus a keyword match for numbers and clause words.
3. If the best match is below a similarity threshold, answer "not found in your document" without calling the model.
4. Otherwise send the chunks with page numbers and the question; the model must cite pages and quotes in a fixed JSON format.
5. The backend validates citations against the sent chunks before returning the answer.

No reranking model at the start. Retrieval quality is measured on the golden set first, and a reranker is added only if the tests show it is needed.

6.4 Privacy in AI calls

- Personal identifiers are masked before any text is sent (national ID, iqama, IBAN, phone, email). Masking alone does not anonymize the contract owner, since the contract text can still identify them.
- Gemini is used on the paid tier. Under Google's published terms, paid-tier prompts and responses are not used to improve Google's products; the data is still processed by the provider.

- Review each provider's processing, retention, and legal terms, including data transfer outside Saudi Arabia under the PDPL, before sending real contracts. Name the providers in the privacy policy.
- Send only what each task needs (classification sends the first pages only; chat sends only retrieved chunks).
- Development and testing use sample or anonymized contracts only.

6.5 Cost control v1.3

- Token usage and cost are saved on every analysis and chat message.
- Daily AI budget: $2, set in `BAYANUK_DAILY_AI_BUDGET_USD` and checked by the application before every AI call. It resets at 00:00 Riyadh time (UTC+3) for both the pipeline and chat. Provider spending alerts are not used as a hard stop. Reviewed after the first 50 real analyses.
- When the budget is reached: documents in progress pause (section 5); new uploads are refused with `ai_budget_reached` and do not use an upload; chat questions are refused with `ai_budget_reached` and are not counted. v1.3
- Limits: guests 2 uploads per day per device and IP; users 10 per day in MVP; chat limits per document (guests 3, users 20). Only a new, distinct document uses an upload. v1.3
- Re-uploading the same file (same sha256) by the same owner (the user, or the guest token for a guest) does not create a new document. A `ready`, in-progress, or `rejected` document is returned as it is. A `failed` document is queued again with the same id, up to 2 retries per document; after that the failure is returned. The new copy of the file is not stored. v1.3
- Limited retries; OCR fallback to Gemini only for pages Tesseract cannot read.

7. API (v1) v1.3

This section summarizes the approved integration contract. The exact requests, responses, and error codes are in `docs/integration-contract.md`; both the backend and the frontend mock API follow it. Any change to the contract is approved by Jana before it is applied.

7.1 Conventions

- JSON over HTTPS under `/api/v1`. IDs are UUIDs.
- The frontend sends the interface language in `Accept-Language` (`ar` or `en`); the backend returns error messages in that language.
- Signed-in users use Sanctum SPA cookie sessions. Before the first request that changes data, the frontend calls `GET /sanctum/csrf-cookie`. Every request is sent with credentials, and the `XSRF-TOKEN` cookie value is sent in the `X-XSRF-TOKEN` header. On `419` the frontend refreshes the CSRF cookie and repeats the request once.
- Local development: the backend runs on `http://localhost:8000`, and the Vite dev server proxies `/api` and `/sanctum` to it, so both share one origin. The production domain is set up at deployment.

7.2 Endpoints

Method	Endpoint	Description
POST	`/guest-token`	Issues a new guest token, returned once: `201 { token }`. Only its hash is stored. v1.3

Method	Endpoint	Description
POST	`/documents`	Multipart upload, one field `file`. Returns `202 { id, status }`; for a duplicate, the existing document.
GET	`/documents/{id}/status`	Lightweight status for polling: `{ id, status, status_detail }`. v1.3
GET	`/documents/{id}`	Full result: document, latest analysis, terms, findings, summary.
GET	`/documents/{id}/file`	Signed URL to view the original file, valid 5 minutes: `{ url, expires_at }`. v1.3
DELETE	`/documents/{id}`	Deletes everything immediately. Returns 204.
GET	`/documents`	Signed-in user's documents, paginated: `{ data, meta }`. v1.3
GET	`/documents/{id}/chat`	Chat history and `remaining_questions`.
POST	`/documents/{id}/chat`	Ask a question. Returns answer, citations, answer_found, and `remaining_questions`. v1.3
POST	`/analyses/{id}/feedback`	Thumbs up/down, optional field and comment.
POST	`/comparisons`	(MVP+) `{ document_a_id, document_b_id }`.
GET	`/comparisons/{id}`	(MVP+) Comparison result.
POST	`/auth/magic-link`	Send a sign-in link to an email (rate limited).
POST	`/auth/magic-link/verify`	Exchange the link token for a session; claims guest documents sent in `X-Guest-Token`. POST only. v1.3
POST	`/auth/google/redirect`	Saves the guest token with a random `state` and returns `{ url }` for Google sign-in. v1.3
GET	`/auth/google/callback`	Google returns here; the backend signs in, claims guest documents, and redirects to the frontend `/auth/callback`. v1.3
GET / POST	`/me`, `/auth/logout`	Current user; sign out.
PATCH / DELETE	`/me`	Update locale or keep_documents; delete account.

7.3 Guest token

1. The backend generates the token. Before the first upload, if none is stored, the frontend calls `POST /guest-token`. The backend stores only its SHA-256 hash.
2. The frontend keeps it in `localStorage` and sends it only in the `X-Guest-Token` header (never in the body or the URL) on every document and question request while the user is not signed in.
3. After sign-in, it is sent once in the same header so the guest documents are claimed, then deleted.
4. On `invalid_guest_token`, the frontend deletes it and asks for a new one. A valid token on another guest's document gets `document_not_found`.

7.4 Sign-in

- **Magic link:** the email link opens the frontend page `/auth/verify?token=...`, which sends the token to the verify endpoint with POST, never GET, because email scanners open links on their own and could use up the single-use token. The backend sets the session cookie. Expired, used, or unknown links show a clear message on the same page with an option to request a new link.
- **Google:** the frontend saves the current page, calls `POST /auth/google/redirect`, and opens the returned URL as a full page. After the callback, the backend redirects to `/auth/callback`. On success the frontend deletes the guest token, loads `/me`, and returns to the saved page. On failure it gets `?error=google_cancelled`, `google_invalid_state`, or `google_sign_in_failed` and offers to try again.

7.5 Status, polling, and progress

- The frontend polls the status every 2 seconds. It stops on `ready`, `failed`, or `rejected`, or on `404` or `410`. While `status_detail` is `ai_budget_reached`, it polls every 60 seconds instead.
- The nine statuses map to the four progress stages in the MVP Definition (section 5):

Status	Stage shown	Note
`uploaded`, `scanning`	Checking file	While `uploaded`, a queue message shows under the stages.
`extracting`	Reading document	
`classifying`, `analyzing`	Finding key terms	
`indexing`	Looking for points that need attention	
`ready` / `rejected` / `failed`	Result / not supported / could not analyze	The reason comes from `status_detail`.

status_detail	With status	Meaning
`unsafe_file`	rejected	The file failed the safety scan.
`too_many_pages`	rejected	More than 30 pages.
`not_employment_document`	rejected	Not a job offer or employment contract.
`file_unreadable`	failed	The text could not be read.
`analysis_failed`	failed	The analysis failed after its retry.
`ai_budget_reached`	any processing status	Paused until the daily AI budget resets.

7.6 Errors

Every error has one shape. `code` is the same in both languages and is compared as an exact string; `message` is translated by the backend and safe to show; `details` is used only by `validation_error`.

```
{ "error": { "code": "file_too_large", "message": "The file is larger than the allowed limit (10 MB)." } }
```

Status	Code	When
400	`invalid_guest_token`	Guest token missing or not in the right format.
400	`link_invalid`	Sign-in link token unknown or changed.
401	`unauthorized`	Not signed in on a request that needs sign-in.
404	`document_not_found`	Document does not exist or is not the user's (one code for both).
410	`document_expired`	Document deleted after the retention period.
410	`link_expired`, `link_used`	Sign-in link expired or already used.
419	`csrf_token_mismatch`	CSRF token expired; refresh and repeat once.
422	`file_type_not_supported`, `file_too_large`	Upload is not PDF or DOCX, or is larger than 10 MB.
422	`validation_error`	A field is missing or not valid; `details` lists messages per field.
429	`upload_limit_reached`	Daily upload limit reached.
429	`question_limit_reached`	Question limit for the document reached.
429	`ai_budget_reached`	Upload or question while the daily AI budget is reached.
429	`magic_link_limit_reached`	Too many sign-in link requests.
429	`too_many_requests`	Any other rate limit.
500	`server_error`	Unexpected error; the message never shows internal details.

A rejected or failed document is not an upload error: the document is created, and the reason arrives through the status (section 7.5).

7.7 Example response: GET /documents/{id}

```
{
  "id": "0193...", "status": "ready", "document_type": "employment_contract",
  "language": "ar", "page_count": 12, "expires_at": "2026-11-20T10:00:00Z",
  "analysis": {
    "id": "0193...", "summary": "...", "summary_language": "en",
    "terms": [
      { "key": "compensation.basic_salary", "value": 8000, "unit": "SAR",
        "found": true, "page": 3, "quote": "..." },
      { "key": "notice.employer_days", "found": false }
    ],
    "findings": [
      { "category": "non_compete", "severity": "attention", "title": "Non-compete clause",
        "explanation": "...", "page": 9, "quote": "..." }
    ]
  }
}
```

8. Security

Risk	Control
Malicious files	Type checked by content, size and page limits, ClamAV scan, files never executed or served from the API domain directly.
Access to other people's documents	UUID IDs plus DocumentPolicy on every route (owner user or matching guest token hash). Guest tokens are random values issued by the backend; only their SHA-256 hash is stored. The same `404` for a missing document and another owner's document. Tests for every endpoint. v1.3
Sign-in	Single-use, expiring magic links verified by POST only; rate limits on link requests; Google `state` checked on the callback; Sanctum CSRF protection on every request that changes data. v1.3
File exposure	Private bucket; signed URLs valid 5 minutes; storage keys are random.
Data at rest	Encrypted disks on the server; RustFS server-side encryption; database backups encrypted.
Data in transit	HTTPS only (Cloudflare + nginx), HSTS, secure and SameSite cookies.
Abuse and cost attacks	Rate limits per IP, guest token, and user; daily upload limits; daily AI budget; Cloudflare WAF and bot protection on upload.
Prompt injection inside documents	Document text is passed as data inside clear delimiters; the model cannot call tools; outputs are schema-validated and quotes verified.
Secrets	API keys and other secrets in server environment variables only; never in the repository; separate keys for staging and production.
Personal data	PII masking before AI calls; only needed text sent; paid plans; retention and deletion jobs; no document content in logs; error messages never show internal details. v1.3
Dependencies	composer audit and npm audit in the Jenkins pipeline.

9. Testing strategy

- Backend: Pest feature tests for every endpoint (including access by the wrong owner, and each error code in the contract), unit tests for rules, PII masking, numeral normalization, and quote verification. AI calls are faked in tests. v1.3
- Golden set: 30 sample documents (15 Arabic, 15 English; offers and contracts; including 5 scanned) written or anonymized by the team, each with an expected JSON. A command (`php artisan bayanuk:eval`) runs the real pipeline and reports accuracy per field. Run before every prompt or model change.
- Before launch: retrieval quality (decides if a reranker is needed), time to result, chat answer time, scanned and large files, user isolation, and cost per analysis, as listed in the AI Stack Decision.
- Frontend: Vitest + Testing Library for components (term cards, findings, chat citations); checks in both RTL and LTR. The MSW mock API follows the contract exactly, including error responses, so the same tests keep passing against the real backend. v1.3
- End-to-end: Playwright smoke test on staging: upload, wait, see result, ask a question, delete.
- Manual: mobile check on real phones in Arabic and English before each release.

10. Environments and deployment

Environment	Where	Purpose
Local	Docker Compose on each laptop; Vite dev server proxying `/api` and `/sanctum` to the backend v1.3	Development with fake AI, or real APIs with sample contracts only.
Staging	Studio server, separate containers and database	Auto-deployed from `development`; team testing and demos. v1.3
Production	Studio server (later: dedicated server if load grows)	Deployed from `main` after approval.

Jenkins pipeline

1. Install dependencies; run linters (Pint, ESLint) and type checks (PHPStan level 6, tsc).
2. Run tests (Pest, Vitest); run composer audit and npm audit.
3. Build Docker images; tag with commit SHA.
4. Deploy to staging automatically (`development`) or to production on manual approval (`main`). v1.3
5. Run migrations, restart Horizon workers, run the Playwright smoke test.
6. On failure, roll back to the previous image tag.

Docker Compose services

api (PHP-FPM), horizon (queue workers, 2 to 4 processes), scheduler (retention job), postgres (with pgvector), redis, rustfs, clamav, behind the host nginx. No AI model containers. Daily database backups to a separate disk, kept 7 days.

11. Observability

Signal	Where	Alert when
Pipeline failures	Horizon + Prometheus counter	More than 5% failed in 1 hour.
Queue wait time	Horizon metrics	Waiting over 2 minutes.
Time to result	processing_finished_at minus created_at	90th percentile over 90 seconds.
AI cost	usage_events, daily Grafana panel	Daily cost reaches 80% of the $2 budget.
AI budget pauses	Documents with `status_detail = ai_budget_reached`	Any document paused by the budget. v1.3
AI provider errors	Laravel logs and Horizon failed jobs	Error rate from Gemini or Voyage AI rises.
Errors	Laravel logs (no document content)	Any 5xx spike.
Server health	Prometheus node exporter	Disk over 80%, memory pressure.

12. Technical risks

Risk	Impact	Mitigation
Arabic PDF text comes out reversed or broken	Wrong extraction	Detect garbled text (ratio checks); fall back to OCR, then to Gemini for that page; include such files in the golden set.
PHP is weaker than Python for document processing	Slower progress	Use proven command-line tools (poppler, Tesseract) called from Laravel; if needed, add a small Python extraction service later without changing the API.
AI returns wrong numbers	Loss of trust	Quotes verified, totals recomputed, every value links to its page, user feedback per field.
AI provider outage or price change	Downtime or cost	Provider abstraction; queue retries; backup provider still to be decided.
Gemini price doubles in January 2027	Cost	About $0.042 per analysis; the $2 budget covers about half as many analyses.
Cost abuse through uploads	Unexpected bills	Limits, daily AI budget, Cloudflare protection.
Frontend and backend drift apart	Rework when connecting v1.3	One integration contract in the repository; mocks and endpoints follow it; changes approved by Jana first.
Contract data processed outside Saudi Arabia	Legal	Review provider terms and PDPL before real contracts; state providers in the privacy policy.
Single server	Downtime	Backups, documented restore steps tested once before launch.